3.3. PROMISE

3.3.1. Giới thiệu về Promise

Promise là một cơ chế trong **JavaScript** giúp bạn thực thi các tác vụ bất đồng bộ mà không rơi vào callback hell hay pyramid of doom (là tình trạng các hàm callback lồng vào nhau ở quá nhiều tầng).

Để tạo ra một **promise object** thì bạn dùng **class Promise** có sẵn trong trình duyệt như sau:

```
const promise = new Promise(
    /* executor */ function (resolve, reject) {
        // Thực thi các tác vụ bất đồng bộ ở đây, và gọi `resolve(result)` khi tác
        // vụ hoàn thành. Nếu xảy ra lỗi, gọi đến `reject(error)`.
    },
)
```

Trong đó, **executor** là một hàm có hai tham số:

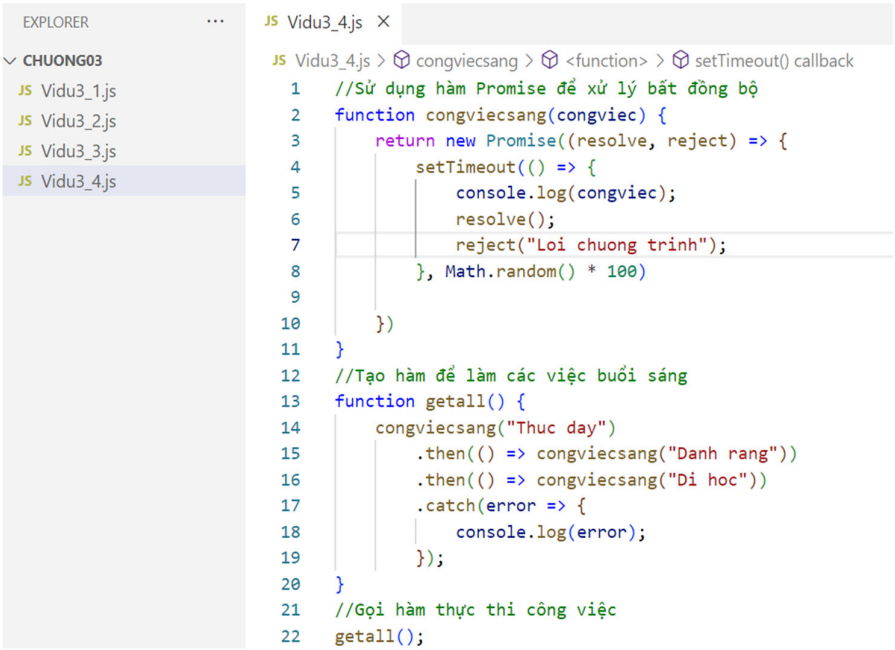
- resolve là hàm sẽ được gọi khi promise hoàn thành
- reject là hàm sẽ được gọi khi có lỗi xảy ra

Promise cung cấp **.then()** thực thi một hàm, một Promise hay một Object sau khi code bất đồng bộ được thực hiện thành công và **.catch()** xử lý sau khi mã bất đồng bộ thất bại.

3.3.2. Ví dụ về Promise

Ví dụ 3.4: Sử dụng Promise trong Nodejs

- Tạo tập tin **Vidu3_4.js** trong thư mục **Chuong03**, viết Code như bên dưới và biên dịch:



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\BTTH_Nodejs_Khuong\Chuong03> node Vidu3_4
Thuc day
Danh rang
Di hoc

> OUTLINE
```

Nó được gọi là một chuỗi Promise, congviecsang() sẽ trả về một Promise và được gửi tới hàm congviecsang() tiếp theo trong chuỗi.

3.3.3. Promise all

Trong trường hợp chúng ta cần kết quả là tập hợp kết quả của nhiều **async function**, chúng ta sẽ dùng **promise.all().** Promise.all() còn ngăn chặn tình trạng Promise hell với những thao tác phức tạp.

Promise.all() trả về kết quả là một **array**, là kết quả của các promise con trong nó.

Promise.all() sẽ resolve khi tất cả các promise trong nó đều thành công. Ngược lại chỉ cần một cái lỗi thì sẽ reject().

Promise.all() thường được sử dụng khi gọi nhiều API cùng lúc hoặc gửi email hàng loạt,...

Ví dụ 3.5: Ví dụ về Promise all

- Tạo thư mục **content** trong thư mục **Chuong03**, tạo 2 tập tin trong thư mục **content** là: tập tin **file1.txt** có nội dung: "Vi du ve Promise all", tập tin **file 2.txt** có nội dung: "Giang vien: ThS. Nguyen Van Khuong"

```
CHUONG03
  content
    file1.txt
    file2.txt
```

```
content > file2.txt
1   Giang vien: ThS. Nguyen Van Khuong
```

- Tạo tập tin **Vidu3_5.js** trong thư mục **Chuong03**, viết Code như bên dưới và biên dịch:

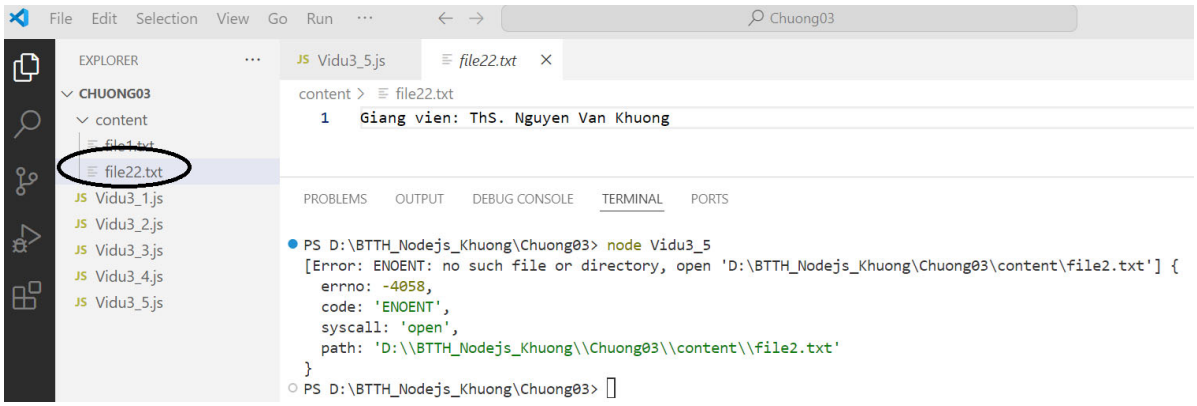
```
EXPLORER
CHUONG03
  content
    file1.txt
    file2.txt
  Vidu3_1.js
  Vidu3_2.js
  Vidu3_3.js
  Vidu3_4.js
  Vidu3_5.js
```

```
Vidu3_5.js
Vidu3_5.js > ...
1   const { error } = require('console');
2   const fs = require('fs');
3
4   function readFilePromise(path) {
5       return new Promise((resolve, reject) => {
6           fs.readFile(path, 'utf-8', (err, value) => {
7               if (err) {
8                   reject(err);
9               } else {
10                  resolve(value);
11              }
12          })
13      });
14  }
15
```

```
16  Promise.all([
17      readFilePromise('./content/file1.txt'),
18      readFilePromise('./content/file2.txt')
19  ])
20      .then((values) => {
21          console.log(values);
22      })
23      .catch((error) => {
24          console.log(error);
25      })
```

- Biên dịch chương trình ta sẽ được:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\BTTH_Nodejs_Khuong\Chuong03> node Vidu3_5
[ 'Vi du ve Promise all', 'Giang vien: ThS. Nguyen Van Khuong' ]
PS D:\BTTH_Nodejs_Khuong\Chuong03>
```

- Bây giờ giả dụ thay đổi tên tập tin file2.txt thành file22.txt, biên dịch lại sẽ bị lỗi và không đọc bất kì tập tin nào: